



Bitácora S07 - Mexi Banco: de un monolito a microservicios

Paso 1: La historia

Al separar la aplicación monolítica en cinco microservicios independientes, se perdió la garantía transaccional local (`@Transactional`). Anteriormente, si una operación como el abono fallaba, la base de datos deshacía automáticamente los cambios porque todo residía en un único proceso y contexto de memoria. Al migrar a comunicación HTTP, cada servicio procesa y confirma únicamente su propia base de datos; si se producen fallos de red a mitad de una transacción compuesta, existe el riesgo de dejar el sistema en un estado inconsistente.

Paso 2: Directorio (Eureka)

En la configuración inicial, el tablero de Eureka en el puerto 8761 no respondía. Tras habilitar el servidor de descubrimiento y configurar los clientes en cada módulo, el tablero logró registrar exitosamente a los cinco servicios del sistema.

Respuesta "Piensa": Aunque los microservicios ya reportan su existencia al directorio, el servicio de **transferencia** no lo utiliza de forma directa para localizar al servicio de **cuenta**. Continúa empleando una dirección estática (`http://cuenta:8080`) definida en las variables de entorno del archivo `compose`, delegando la resolución de nombres al DNS interno de Docker en lugar de consultar la agenda dinámica de Eureka.

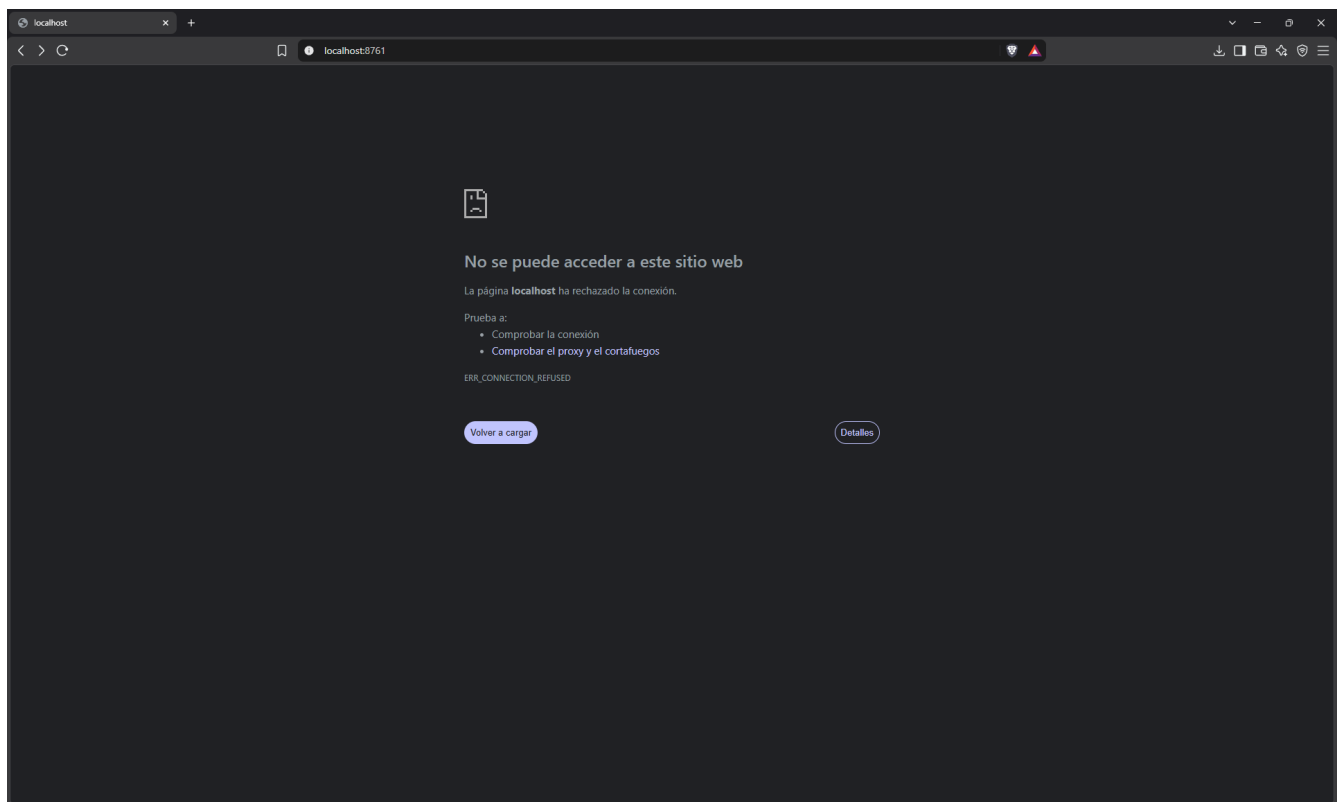


Figure 1: Tablero de Eureka antes de encender el servicio de descubrimiento.

System Status

Environment	test	Current time	2026-10-10T00:30:57 +0000
Data center	default	Uptime	00:00
		Lease expiration enabled	true
		Renews threshold	10
		Renews (last min)	0

THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CUENTA	n/a (1)	(1)	UP (1) - 6cc9ad793c4d9.cuenta.8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 9082e6241860.movimiento.8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 200f6ee632d9b.notificacion.8080
SPEI	n/a (1)	(1)	UP (1) - d8dfe17cd8d8.spei.8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 9d3d9ce258a9.transferencia.8080

General info

Name	Value
total-avail-memory	81mb
num-of-cpus	20
current-memory-usage	38mb (46%)
server-up-time	00:00
registered-replicas	http://localhost:8761/eureka/
unavailable-replicas	http://localhost:8761/eureka/
available-replicas	

Instance info

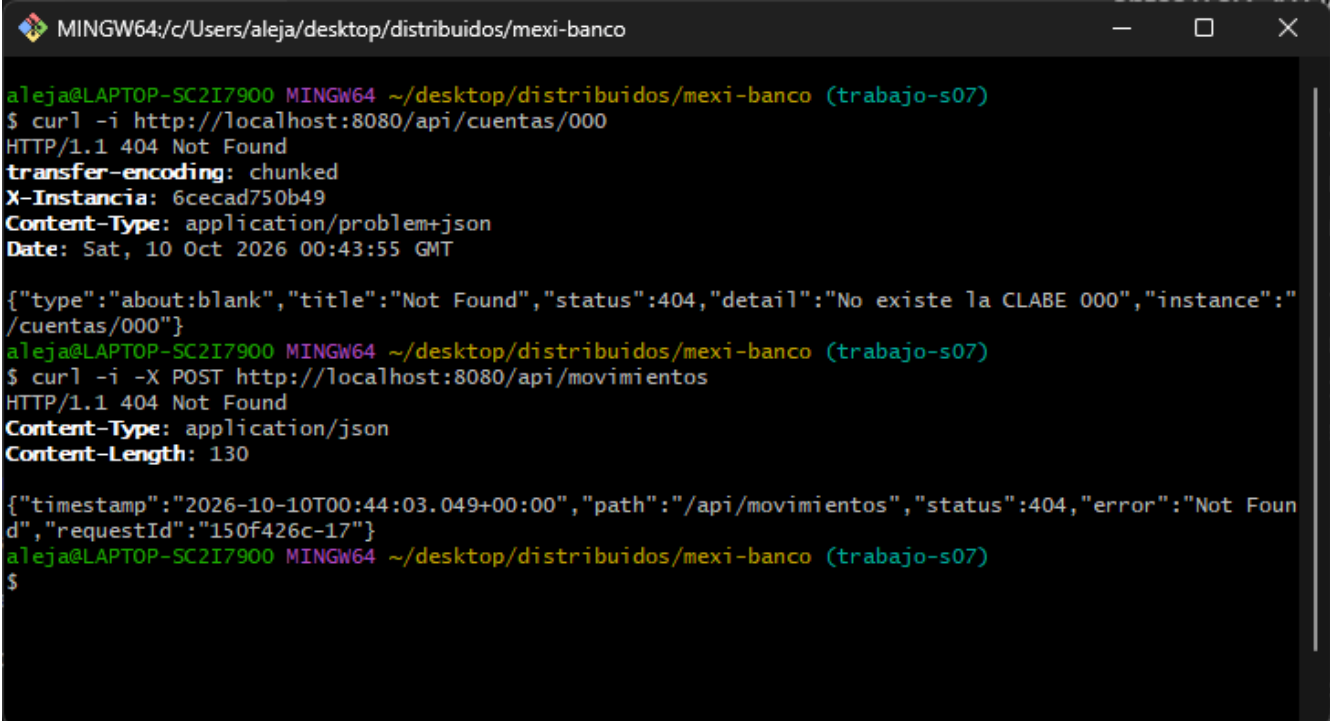
Name	Value
ipAddr	172.18.0.2
status	UP

Figure 2: Tablero de Eureka mostrando los cinco servicios registrados correctamente y activos.

Paso 3: Gateway

El API Gateway funciona como la única puerta de entrada al sistema, centralizando las reglas de control de acceso. Las pruebas de acceso directo al puerto interno 8082 devolvieron un código de error 400. Al enrutar las peticiones a través del puerto 8080 del gateway, se obtuvieron dos respuestas con código 404 motivadas por razones distintas:

- El primer error 404 provino directamente del **servicio de cuenta** (evidenciado por la presencia de la cabecera **X-Instancia**), confirmando que la ruta y el microservicio existen, pero el número de cuenta consultado no fue hallado.
- El segundo error 404 fue emitido por el **Gateway**, el cual bloqueó e interrumpió la petición debido a que no existe una regla de enrutamiento pública expuesta para aceptar solicitudes POST hacia los endpoints internos de movimientos.

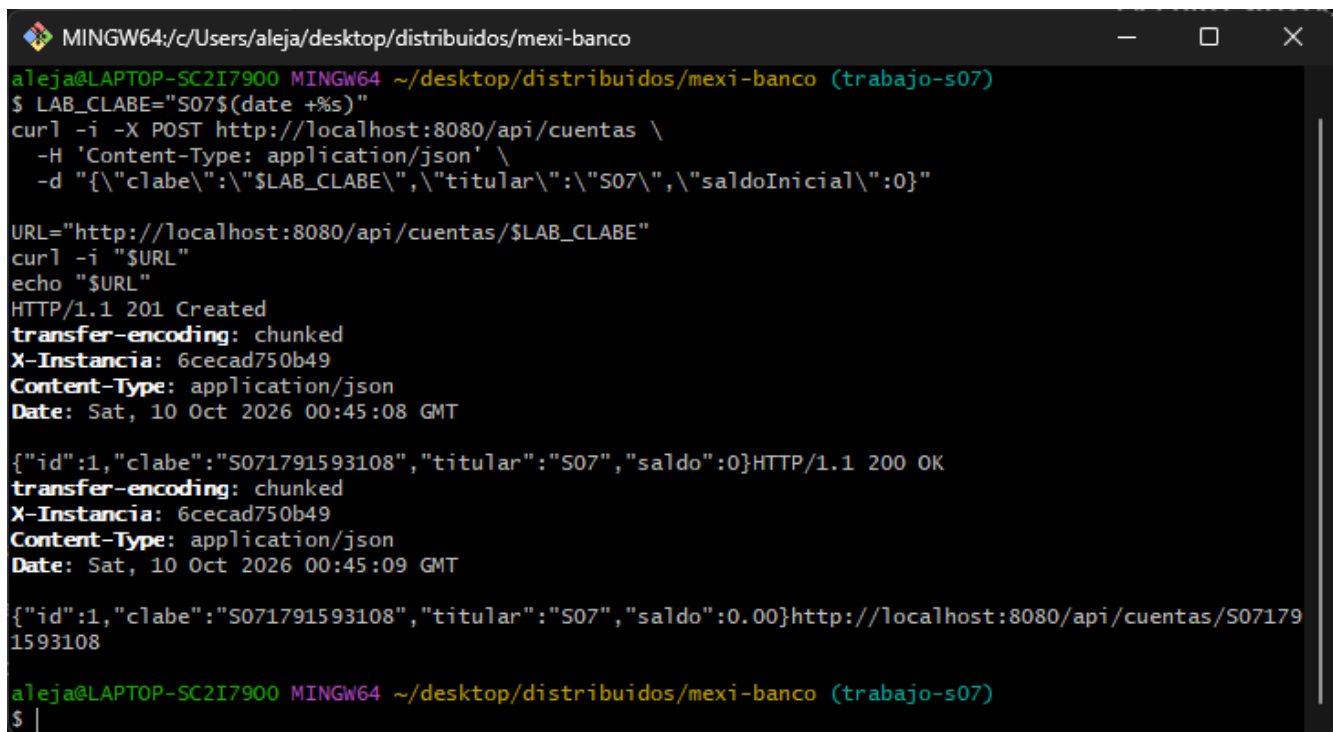


```
MINGW64:/c/Users/aleja/desktop/distribuidos/mexi-banco
aleja@LAPTOP-SC2I7900 MINGW64 ~/desktop/distribuidos/mexi-banco (trabajo-s07)
$ curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: 6cecad750b49
Content-Type: application/problem+json
Date: Sat, 10 Oct 2026 00:43:55 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}
aleja@LAPTOP-SC2I7900 MINGW64 ~/desktop/distribuidos/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 130

{"timestamp":"2026-10-10T00:44:03.049+00:00","path":"/api/movimientos","status":404,"error":"Not Found","requestId":"150f426c-17"}
aleja@LAPTOP-SC2I7900 MINGW64 ~/desktop/distribuidos/mexi-banco (trabajo-s07)
$
```

Figure 3: Prueba de errores 404 devueltos por el gateway y validaciones de seguridad.



```
MINGW64:/c/Users/aleja/desktop/distribuidos/mexi-banco
aleja@LAPTOP-SC2I7900 MINGW64 ~/desktop/distribuidos/mexi-banco (trabajo-s07)
$ LAB_CLABE="S07$(date +%s)"
curl -i -X POST http://localhost:8080/api/cuentas \
  -H 'Content-Type: application/json' \
  -d '{"clabe":"$LAB_CLABE","titular":"S07","saldoInicial":0}'

URL="http://localhost:8080/api/cuentas/$LAB_CLABE"
curl -i "$URL"
echo "$URL"
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 6cecad750b49
Content-Type: application/json
Date: Sat, 10 Oct 2026 00:45:08 GMT

{"id":1,"clabe":"S071791593108","titular":"S07","saldo":0}HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 6cecad750b49
Content-Type: application/json
Date: Sat, 10 Oct 2026 00:45:09 GMT

{"id":1,"clabe":"S071791593108","titular":"S07","saldo":0.00}http://localhost:8080/api/cuentas/S071791593108
aleja@LAPTOP-SC2I7900 MINGW64 ~/desktop/distribuidos/mexi-banco (trabajo-s07)
$ |
```

Figure 4: Validación de registro exitoso (201 Created) y consulta de saldo (200 OK) a través de la puerta única.

Pasos 4 y 5: Copias y caída

Tras escalar horizontalmente el microservicio de cuentas, el panel de administración reflejó **CUENTA UP (3)**. Al emitir peticiones simultáneas, las cabeceras **X-Instancia** demostraron que el sistema distribuyó la carga mediante un algoritmo *Round Robin* equitativo entre las tres réplicas disponibles.

Simulación de caída de nodo: A las 19:15:13 se interrumpió abruptamente el contenedor de la primera instancia de cuenta (`docker kill`). Durante el intervalo posterior, se registraron de forma intermitente códigos de error **500 y 503** en las peticiones concurrentes hasta las 19:15:36, momento en el cual el tráfico se estabilizó al 100% de éxito.

¿Por qué ocurrieron estos errores? Este comportamiento se debe al desfase temporal inherente en la propagación de eventos distribuidos. El directorio Eureka requiere el vencimiento del plazo de latidos (*heartbeats*) configurado en 15 segundos para declarar inactiva a una instancia, y de forma paralela el gateway opera con una caché interna de rutas. Durante esa ventana temporal de aproximadamente 23 segundos, el balanceador continuó enviando llamadas hacia una dirección de contenedor ya inexistente.

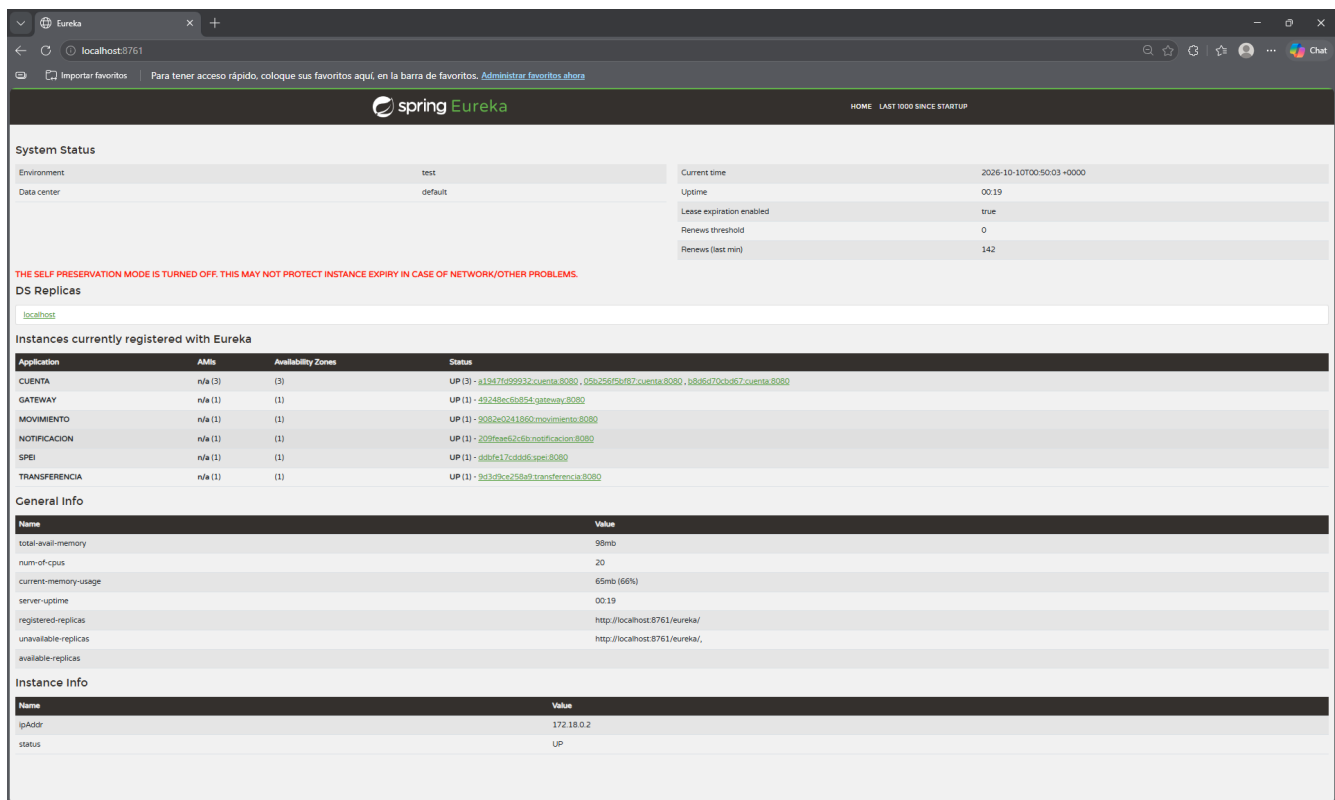


Figure 5: Panel de Eureka mostrando el escalamiento a tres instancias activas de CUENTA.

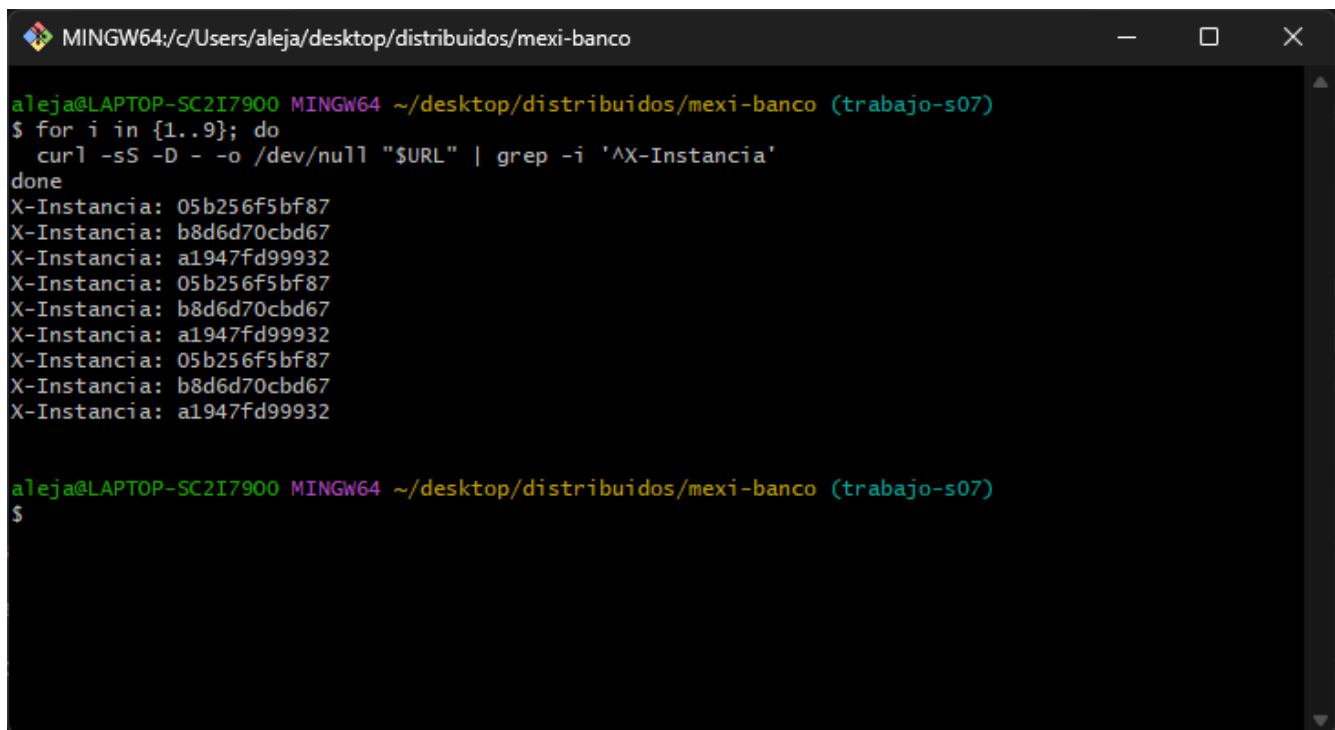
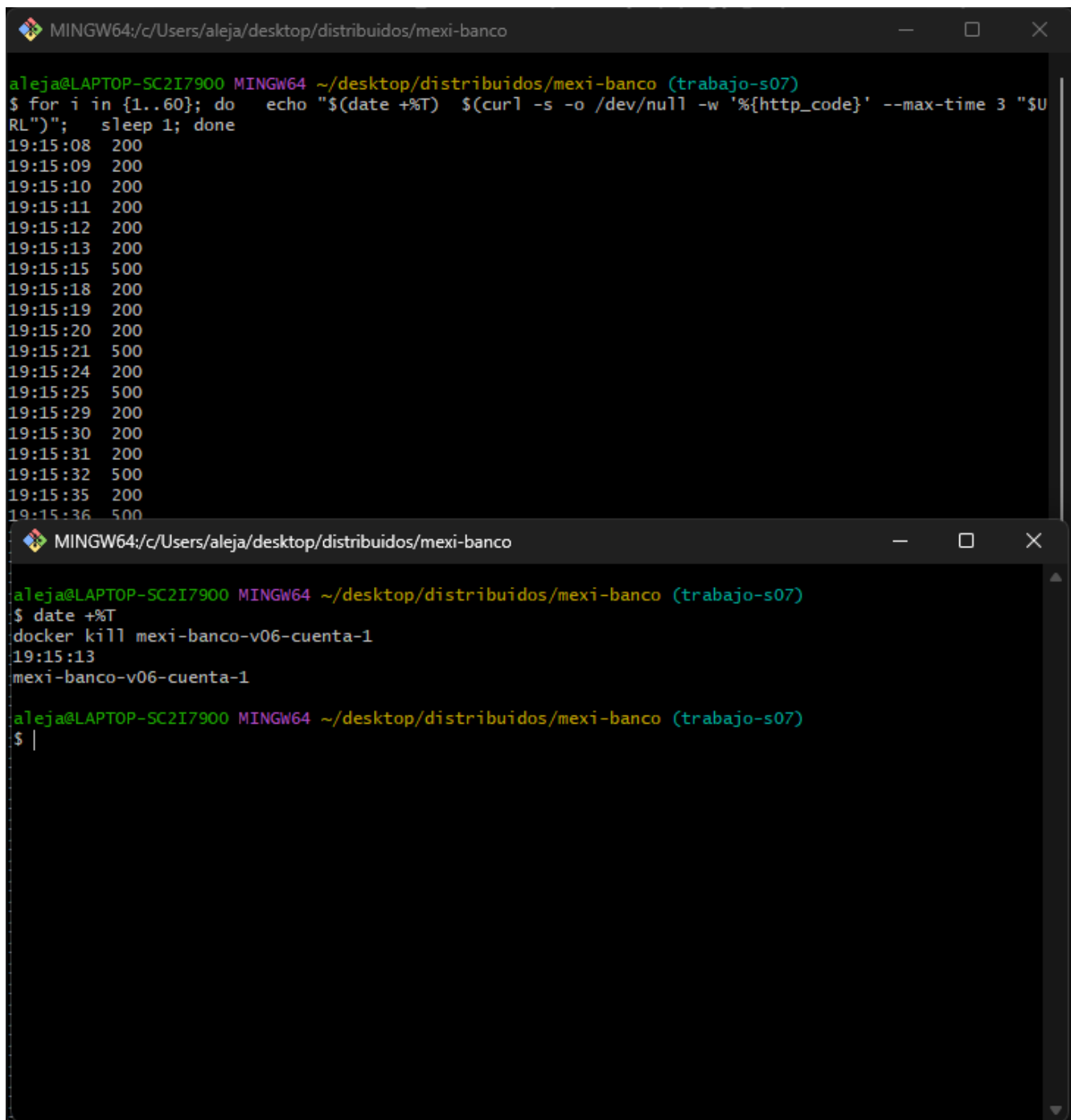


Figure 6: Validación del balanceo de carga por turnos mediante la cabecera X-Instancia.



The image shows two terminal windows from a MINGW64 environment. The top window displays a script that monitors HTTP status codes over time, showing a mix of 200 and 500 status codes. The bottom window shows the user killing a Docker container named 'mexi-banco-v06-cuenta-1' and then checking its status, which shows as 'exited'.

```
MINGW64:/c/Users/aleja/desktop/distribuidos/mexi-banco
aleja@LAPTOP-SC2I7900 MINGW64 ~/desktop/distribuidos/mexi-banco (trabajo-s07)
$ for i in {1..60}; do echo "$(date +%T) $(curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$U
RL")"; sleep 1; done
19:15:08 200
19:15:09 200
19:15:10 200
19:15:11 200
19:15:12 200
19:15:13 200
19:15:15 500
19:15:18 200
19:15:19 200
19:15:20 200
19:15:21 500
19:15:24 200
19:15:25 500
19:15:29 200
19:15:30 200
19:15:31 200
19:15:32 500
19:15:35 200
19:15:36 500

MINGW64:/c/Users/aleja/desktop/distribuidos/mexi-banco
aleja@LAPTOP-SC2I7900 MINGW64 ~/desktop/distribuidos/mexi-banco (trabajo-s07)
$ date +%T
19:15:13
docker kill mexi-banco-v06-cuenta-1
mexi-banco-v06-cuenta-1
aleja@LAPTOP-SC2I7900 MINGW64 ~/desktop/distribuidos/mexi-banco (trabajo-s07)
$ |
```

Figure 7: Monitoreo en tiempo real de códigos HTTP mostrando intermitencias de error tras la caída del nodo.

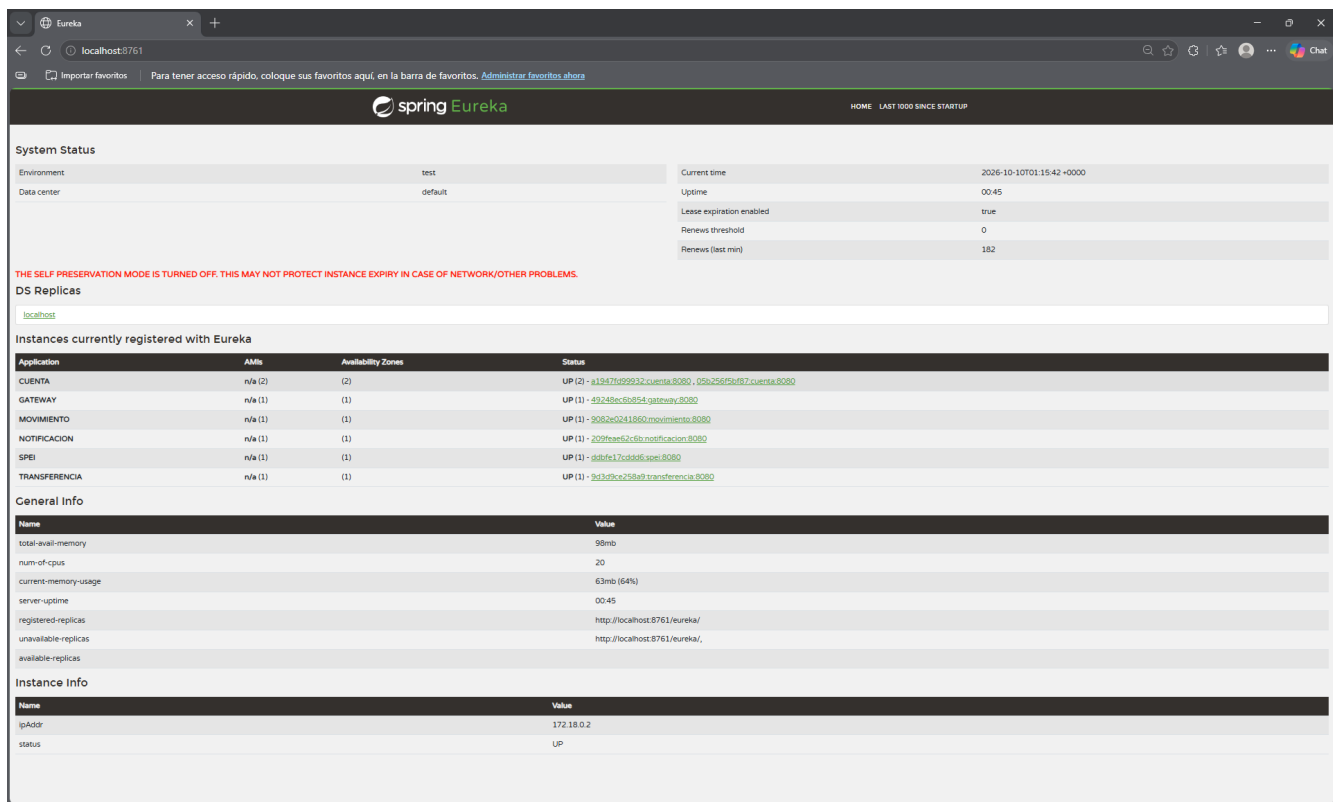


Figure 8: Registro de la hora exacta del comando de terminación y recuperación de estabilidad en el sistema.

Cierre

- El **directorio** resuelve la localización dinámica de servicios activos y sus direcciones IP, erradicando el acoplamiento estricto de direcciones estáticas.
- El **gateway** resuelve la consolidación de un punto único de acceso perimetral, resguardando y aislando los componentes lógicos internos de la red exterior.
- El **balanceador** resuelve la distribución equitativa de peticiones entre múltiples réplicas y tolera fallos mitigando el envío de tráfico a nodos caídos.
- **Durante la caída de una réplica se observaron** errores intermitentes 500/503 por espacio de 23 segundos debido al tiempo de expiración de los latidos en el directorio y la vigencia de la caché del gateway.

Tarea: Transferencia balanceada

Se configuró el cliente interno de transferencias implementando la anotación `@LoadBalanced` en el contexto de Spring. De este modo, el microservicio de transferencia consulta de manera dinámica al directorio

Eureka para resolver el nombre lógico `http://cuenta`. La ejecución exitosa del script de demostración demostró que las distintas copias atendieron de forma equitativa las operaciones de cargos y abonos.

```
MINGW64/c/Users/aleja/desktop/distribuidos/mexi-banco
aleja@LAPTOP-SC2I7900 MINGW64 ~/desktop/distribuidos/mexi-banco (trabajo-s07)
$ ./demo-v06.sh

== 0. Esperar a que el directorio conozca a todos y la puerta encuentre a cuenta ==
-> listo en 1 s. Lo que sabe el directorio (http://localhost:8761/eureka/apps):
CUENTA      3 instancia(s)
GATEWAY     1 instancia(s)
MOVIMIENTO  1 instancia(s)
NOTIFICACION 1 instancia(s)
SPEI        1 instancia(s)
TRANSFERENCIA 1 instancia(s)
-> 404 de cuenta (no 503 del gateway) para una CLABE que no existe: la puerta ya sabe llegar

== 1. Abrir la cuenta de Ana con 1000 (por la puerta: POST /api/cuentas) ==
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clave":"002180009159718701","titular":"Ana","saldoInicial":1000}'
[HTTP 201]
-> el gateway quito /api y lo mando a lb://cuenta: a una de sus tres replicas

== 2. Abrir la cuenta de Beto con 500 (misma puerta) ==
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clave":"002180009159718702","titular":"Beto","saldoInicial":500}'
[HTTP 201]

== 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ==
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009159718701","claveDestino":"002180009159718702","monto":200,"fecha":"2026-10-10T01:53:10.8852653Z"}'
[HTTP 201]
-> transferencia llamo a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:
cuenta-2 GET /cuentas/002180009159718701 atendida por a1947fd99932
cuenta-2 GET /cuentas/002180009159718702 atendida por a1947fd99932
cuenta-2 POST /cuentas/002180009159718701/cargos atendida por a1947fd99932
cuenta-2 POST /cuentas/002180009159718702/abonos atendida por a1947fd99932
-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia

== 4. Consultar esa transferencia ==
$ curl http://localhost:8080/api/transferencias/1
{"id":1,"claveOrigen":"002180009159718701","claveDestino":"002180009159718702","monto":200.00,"fecha":"2026-10-10T01:53:10.885266Z"}
[HTTP 200]

== 5. Saldos: el dinero no se crea ni se pierde ==
$ curl http://localhost:8080/api/cuentas/002180009159718701
{"id":2,"clave":"002180009159718701","titular":"Ana","saldo":800.00}
[HTTP 200]
$ curl http://localhost:8080/api/cuentas/002180009159718702
{"id":3,"clave":"002180009159718702","titular":"Beto","saldo":700.00}
[HTTP 200]
-> Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700

== 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) ==
$ curl http://localhost:8080/api/movimientos?clave=002180009159718701
[{"id":3,"claveCuenta":"002180009159718701","tipo":"TRANSFERENCIA_ENVIADA","monto":-200.00,"saldoResultante":800.00,"fecha":"2026-10-10T01:53:10.869808Z","referencia":"a 002180009159718702"},{"id":1,"claveCuenta":"002180009159718701","tipo":"DEPOSITO","monto":1000.00,"saldoResultante":1000.00,"fecha":"2026-10-10T01:53:08.722256Z","referencia":"saldo inicial"}]
[HTTP 200]
$ curl http://localhost:8080/api/notificaciones?clave=002180009159718702
[{"id":2,"claveCuenta":"002180009159718702","mensaje":"TRANSFERENCIA_RECIBIDA por 200, saldo resultante 700.00","fecha":"2026-10-10T01:53:11.432229Z"}]
[HTTP 200]

== 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ==
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91597187' -d '{"claveOrigen":"002180009159718701","bancoDestino":"BANCO-X","claveDestino":"012180000000000009","monto":50,"estado":"ENVIADO"}'
[HTTP 201]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91597187' -d '{"claveOrigen":"002180009159718701","bancoDestino":"BANCO-X","claveDestino":"012180000000000009","monto":50,"estado":"ENVIADO"}'
{"id":1,"idempotencyKey":"spei-demo-91597187","claveOrigen":"002180009159718701","bancoDestino":"BANCO-X","claveDestino":"012180000000000009","monto":50.00,"estado":"ENVIADO"}
[HTTP 200]
```

Figure 9: Inicialización y verificación de registros en el directorio durante la ejecución de la demo.

```

MINGW64: c:/Users/aleja/desktop/distribuidos/mexi-banco
01218000000000009", "monto": 50}
{"id": 1, "idempotencyKey": "spei-demo-91597187", "claveOrigen": "002180009159718701", "bancoDestino": "BANCO-X", "claveDestino": "012180000000000009", "monto": 50.00, "estado": "ENVIADO"}
[HTTP 201]
-> mismo id (1 -> 1): no se cobro dos veces, Saldo de Ana: 750.00 (esperado 750)

=== 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ===
consulta 1 X-Instancia: b86d70cbd67 (mexi-banco-v06-cuenta-1)
consulta 2 X-Instancia: a1947fd99932 (mexi-banco-v06-cuenta-2)
consulta 3 X-Instancia: 05b256f5bf87 (mexi-banco-v06-cuenta-3)
consulta 4 X-Instancia: b86d70cbd67 (mexi-banco-v06-cuenta-1)
consulta 5 X-Instancia: a1947fd99932 (mexi-banco-v06-cuenta-2)
consulta 6 X-Instancia: 05b256f5bf87 (mexi-banco-v06-cuenta-3)
consulta 7 X-Instancia: b86d70cbd67 (mexi-banco-v06-cuenta-1)
consulta 8 X-Instancia: a1947fd99932 (mexi-banco-v06-cuenta-2)
consulta 9 X-Instancia: 05b256f5bf87 (mexi-banco-v06-cuenta-3)
-> 3 replicas distintas en 9 consultas; 0 veces se repitio la misma dos veces seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el 8080

=== 9. Los endpoints internos no tienen puerta ===
$ curl -X POST http://localhost:8080/api/cuentas/002180009159718701/abonos -H 'Content-Type: application/json' -d '{"monto": 1000000}'
{"timestamp": "2026-10-10T01:53:18.434+00:00", "path": "/api/cuentas/002180009159718701/abonos", "status": 404, "error": "Not Found", "requestId": "07f6e7d8-1123"}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)

=== 10. Errores de negocio, por la puerta, con el mismo codigo que dio cuenta ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen": "002180009159718701", "claveDestino": "002180009159718799", "monto": 10}'
{"type": "about:blank", "title": "Not Found", "status": 404, "detail": "No existe la CLABE 002180009159718799", "instance": "/transferencias", "servicio": "cuenta"}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen": "002180009159718701", "claveDestino": "002180009159718702", "monto": 99999}'
{"type": "about:blank", "title": "Unprocessable Entity", "status": 422, "detail": "Saldo insuficiente en la CLABE 002180009159718701", "instance": "/transferencias", "servicio": "cuenta"}
[HTTP 422]

=== 11. Los saldos no se movieron con los errores ===
-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco

Fin. Para ver cuanto tarda el directorio en enterarse de una caída: ./demo-v06.sh --latido

aleja@LAPTOP-9C2I790Q MINGW64 ~/desktop/distribuidos/mexi-banco (trabajo-s07)
$ ./demo-v06.sh --latido

=== 0. Esperar a que el directorio conozca a todos y la puerta encuentre a cuenta ===
-> listo en 1 s. Lo que sabe el directorio (http://localhost:8761/eureka/apps):
CUENTA      3 instancia(s)
GATEWAY     1 instancia(s)
MOVIMIENTO  1 instancia(s)
NOTIFICACION 1 instancia(s)
SPEI        1 instancia(s)
TRANSFERENCIA 1 instancia(s)
-> 404 de cuenta (no 503 del gateway) para una CLABE que no existe: la puerta ya sabe llegar

=== 1. Abrir la cuenta de Ana con 1000 (por la puerta: POST /api/cuentas) ===
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clave": "002180009159722901", "titular": "Ana", "saldoInicial": 1000}'
{"id": 4, "clave": "002180009159722901", "titular": "Ana", "saldo": 1000}
[HTTP 201]
-> el gateway quito /api y lo mando a lb://cuenta: a una de sus tres replicas

=== 2. Abrir la cuenta de Beto con 500 (misma puerta) ===
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clave": "002180009159722902", "titular": "Beto", "saldoInicial": 500}'
{"id": 5, "clave": "002180009159722902", "titular": "Beto", "saldo": 500}
[HTTP 201]

=== 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen": "002180009159722901", "claveDestino": "002180009159722902", "monto": 200}'
{"id": 2, "claveOrigen": "002180009159722901", "claveDestino": "002180009159722902", "monto": 200, "fecha": "2026-10-10T01:53:51.896372236Z"}
[HTTP 201]
-> transferencia llamo a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:
cuenta-2 GET /cuentas/002180009159722901 atendida por a1947fd99932

```

Figure 10: Creación de cuentas, procesamiento de transferencias y distribución de carga interna entre réplicas.

```

MINGW64: c:/Users/aleja/desktop/distribuidos/mexi-banco

=== 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009159722901","claveDestino":"002180009159722902","monto":200}'
{"id":12,"claveOrigen":"002180009159722901","claveDestino":"002180009159722902","monto":200,"fecha":"2026-10-10T01:53:51.896372236Z"}
[HTTP 201]
-> transferencia llamo a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:
cuenta-2 GET /cuentas/002180009159722901 atendida por a1947fd99932
cuenta-2 GET /cuentas/002180009159722902 atendida por a1947fd99932
cuenta-2 POST /cuentas/002180009159722901/cargos atendida por a1947fd99932
cuenta-2 POST /cuentas/002180009159722902/abonos atendida por a1947fd99932
-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia

=== 4. Consultar esa transferencia ===
$ curl http://localhost:8080/api/transferencias/2
{"id":12,"claveOrigen":"002180009159722901","claveDestino":"002180009159722902","monto":200.00,"fecha":"2026-10-10T01:53:51.8963722Z"}
[HTTP 200]

=== 5. Saldos: el dinero no se crea ni se pierde ===
$ curl http://localhost:8080/api/cuentas/002180009159722901
{"id":14,"clave":"002180009159722901","titular":"Ana","saldo":800.00}
[HTTP 200]
$ curl http://localhost:8080/api/cuentas/002180009159722902
{"id":15,"clave":"002180009159722902","titular":"Beto","saldo":700.00}
[HTTP 200]
-> Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700

=== 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) ===
$ curl http://localhost:8080/api/movimientos?clave=002180009159722901
[{"id":8,"claveCuenta":"002180009159722901","tipo":"TRANSFERENCIA_ENVIADA","monto":-200.00,"saldoResultante":800.00,"fecha":"2026-10-10T01:53:51.865541Z","referencia":"a 002180009159722902"}, {"id":6,"claveCuenta":"002180009159722901","tipo":"DEPOSITO","monto":1000.00,"saldoResultante":1000.00,"fecha":"2026-10-10T01:53:50.637652Z","referencia":"saldo inicial"}]
[HTTP 200]
$ curl http://localhost:8080/api/notificaciones?clave=002180009159722902
[{"id":15,"claveCuenta":"002180009159722902","mensaje":"TRANSFERENCIA_RECIBIDA por 200, saldo resultante 700.00","fecha":"2026-10-10T01:53:51.912367Z"}]
[HTTP 200]

=== 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ===
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91597229' -d '{"claveOrigen":"002180009159722901","bancoDestino":"BANCO-X","claveDestino":"012180000000000009","monto":50}'
{"id":3,"idempotencyKey":"spei-demo-91597229","claveOrigen":"002180009159722901","bancoDestino":"BANCO-X","claveDestino":"012180000000000009","monto":50,"estado":"ENVIADO"}
[HTTP 201]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91597229' -d '{"claveOrigen":"002180009159722901","bancoDestino":"BANCO-X","claveDestino":"012180000000000009","monto":50}'
{"id":3,"idempotencyKey":"spei-demo-91597229","claveOrigen":"002180009159722901","bancoDestino":"BANCO-X","claveDestino":"012180000000000009","monto":50.00,"estado":"ENVIADO"}
[HTTP 201]
-> mismo id (3 -> 3): no se cobro dos veces. Saldo de Ana: 750.00 (esperado 750)

=== 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ===
consulta 1 X-Instancia: 05b256f5bf87 (mexi-banco-v06-cuenta-3)
consulta 2 X-Instancia: a1947fd99932 (mexi-banco-v06-cuenta-2)
consulta 3 X-Instancia: b86d670cbd67 (mexi-banco-v06-cuenta-1)
consulta 4 X-Instancia: b86d670cbd67 (mexi-banco-v06-cuenta-1)
consulta 5 X-Instancia: a1947fd99932 (mexi-banco-v06-cuenta-2)
consulta 6 X-Instancia: 05b256f5bf87 (mexi-banco-v06-cuenta-3)
consulta 7 X-Instancia: b86d670cbd67 (mexi-banco-v06-cuenta-1)
consulta 8 X-Instancia: a1947fd99932 (mexi-banco-v06-cuenta-2)
consulta 9 X-Instancia: 05b256f5bf87 (mexi-banco-v06-cuenta-3)
-> 3 replicas distintas en 9 consultas; 1 veces se repitio la misma dos veces seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el 8080

=== 9. Los endpoints internos no tienen puerta ===
$ curl -X POST http://localhost:8080/api/cuentas/002180009159722901/abonos -H 'Content-Type: application/json' -d '{"monto":1000000}'
{"timestamp":"2026-10-10T01:53:57.273+00:00","path":"/api/cuentas/002180009159722901/abonos","status":404,"error":"Not Found","requestId":"7da05521-1159"}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)

=== 10. Errores de negocio, por la puerta, con el mismo codigo que dio cuenta ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009159722901","claveDestino":"002180009159722999","monto":10}'

```

Figure 11: Validación de consistencia en saldos finales, operaciones SPEI y reglas de negocio.

```

MINGW64/c/Users/aleja/desktop/distribuidos/mexi-banco
{"id":"3","idempotencyKey":"spei-demo-91597229","claveOrigen":"002180009159722901","bancoDestino":"BANCO-X","claveDestino":"012180000000000009","monto":50,"estado":"ENVIADO"}
[HTTP 201]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91597229' -d '{"claveOrigen":"002180009159722901","bancoDestino":"BANCO-X","claveDestino":"012180000000000009","monto":50}'
{"id":"3","idempotencyKey":"spei-demo-91597229","claveOrigen":"002180009159722901","bancoDestino":"BANCO-X","claveDestino":"012180000000000009","monto":50.00,"estado":"ENVIADO"}
[HTTP 201]
-> mismo id (3 -> 3): no se cobro dos veces. Saldo de Ana: 750.00 (esperado 750)

== 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ==
consulta 1 X-Instancia: 05b256f5bf87 (mexi-banco-v06-cuenta-3)
consulta 2 X-Instancia: a1947fd99932 (mexi-banco-v06-cuenta-2)
consulta 3 X-Instancia: b8d6d70cbd67 (mexi-banco-v06-cuenta-1)
consulta 4 X-Instancia: b8d6d70cbd67 (mexi-banco-v06-cuenta-1)
consulta 5 X-Instancia: a1947fd99932 (mexi-banco-v06-cuenta-2)
consulta 6 X-Instancia: 05b256f5bf87 (mexi-banco-v06-cuenta-3)
consulta 7 X-Instancia: b8d6d70cbd67 (mexi-banco-v06-cuenta-1)
consulta 8 X-Instancia: a1947fd99932 (mexi-banco-v06-cuenta-2)
consulta 9 X-Instancia: 05b256f5bf87 (mexi-banco-v06-cuenta-3)
-> 3 replicas distintas en 9 consultas; 1 veces se repitio la misma dos veces seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el 8080

== 9. Los endpoints internos no tienen puerta ==
$ curl -X POST http://localhost:8080/api/cuentas/002180009159722901/abonos -H 'Content-Type: application/json' -d '{"monto":1000000}'
{"timestamp":"2026-10-10T01:53:57.273+00:00","path":"/api/cuentas/002180009159722901/abonos","status":404,"error":"Not Found","requestId":"7da05521-1159"}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)

== 10. Errores de negocio, por la puerta, con el mismo codigo que dio cuenta ==
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009159722901","claveDestino":"002180009159722999","monto":10}'
{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 002180009159722999","instance":"/transferencias","servicio":"cuenta"}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009159722901","claveDestino":"002180009159722902","monto":99999}'
{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"Saldo insuficiente en la CLABE 002180009159722901","instance":"/transferencias","servicio":"cuenta"}
[HTTP 422]

== 11. Los saldos no se movieron con los errores ==
-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco

== 12. El latido: matar una replica de cuenta sin que se despierta ==
-> en el directorio hay 9 instancias de CUENTA; la victima es 05b256f5bf87 (mexi-banco-v06-cuenta-3)
-> docker kill = se va la luz: el proceso no alcanza a darse de baja. Solo dejan de llegar sus latidos.
-> (docker stop seria un apagado ordenado: Spring la marca DOWN al instante y nadie le manda trafico)
t= 0s CUENTA en el directorio: 3 por la puerta: 1 bien, 0 con error
t= 5s CUENTA en el directorio: 3 por la puerta: 4 bien, 1 con error
t= 10s CUENTA en el directorio: 3 por la puerta: 5 bien, 2 con error
t= 15s CUENTA en el directorio: 3 por la puerta: 7 bien, 3 con error
t= 20s CUENTA en el directorio: 3 por la puerta: 8 bien, 4 con error
t= 25s CUENTA en el directorio: 3 por la puerta: 11 bien, 4 con error
t= 30s CUENTA en el directorio: 3 por la puerta: 12 bien, 5 con error
-> el directorio saco a la replica muerta a los 35 s
-> mientras tanto, por la puerta: 13 respuestas bien y 6 con error (el ultimo error a los 31 s)
-> los errores son el gateway mandando peticiones a una replica que ya no existe: el registro es eventualmente consistente
Lo que sabe ahora el directorio:
CUENTA 2 instancia(s)
GATEWAY 1 instancia(s)
MOVIMIENTO 1 instancia(s)
NOTIFICACION 1 instancia(s)
SPEI 1 instancia(s)
TRANSFERENCIA 1 instancia(s)

== 13. Volver a prender la replica y ver cuanto tarda en reaparecer ==
-> mexi-banco-v06-cuenta-3 aparece otra vez en el directorio a los 7 s de docker start (arranque de la JVM + registro)
-> y el gateway le vuelve a mandar peticiones a los 13 s

Fin. Con EUREKA_LATIDO_S=5 EUREKA_CONSULTA_S=5 EUREKA_EXPIRA_S=15 BALANCEADOR_CACHE=5s (Spring Cloud trae latido 30, expiracion 90, consulta 30 y cache 35s).
aleja@LAPTOP-SC2I7900 MINGW64 ~/desktop/distribuidos/mexi-banco (trabajo-s07)

```

Figure 12: Resultado final de la ejecución de `./demo-v06.sh` comprobando el éxito operativo y el balanceo interno.